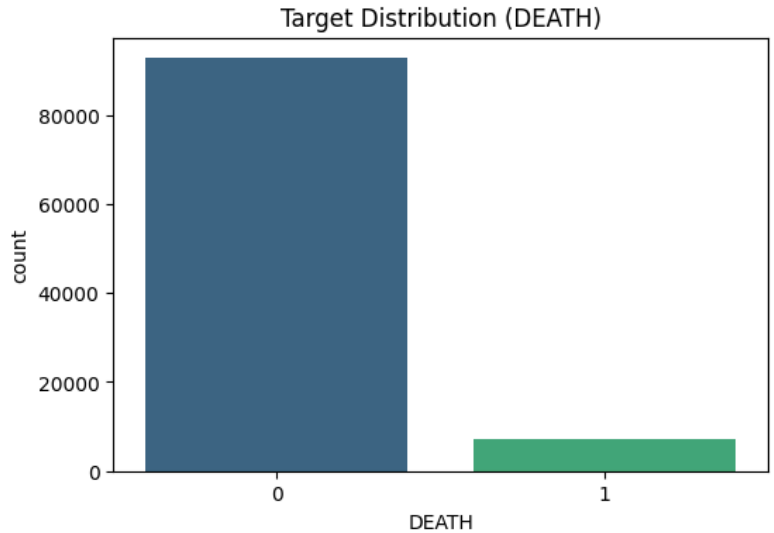
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import json
import shap
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from xgboost import XGBClassifier
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load the full dataset and drop missing values
df_full = pd.read_csv('covid_data.csv').dropna()

# Sample 100,000 rows to optimize training time and memory usage
# random_state is set to 42 for reproducibility as required
df = df_full.sample(n=100000, random_state=42)

# Define the target variable and features
target_col = 'DEATH'
feature_cols = ['AGE', 'SEX', 'PNEUMONIA', 'DIABETES', 'HYPERTENSION', 'OBESITY', 'COPD']

X = df[feature_cols]
y = df[target_col]
```

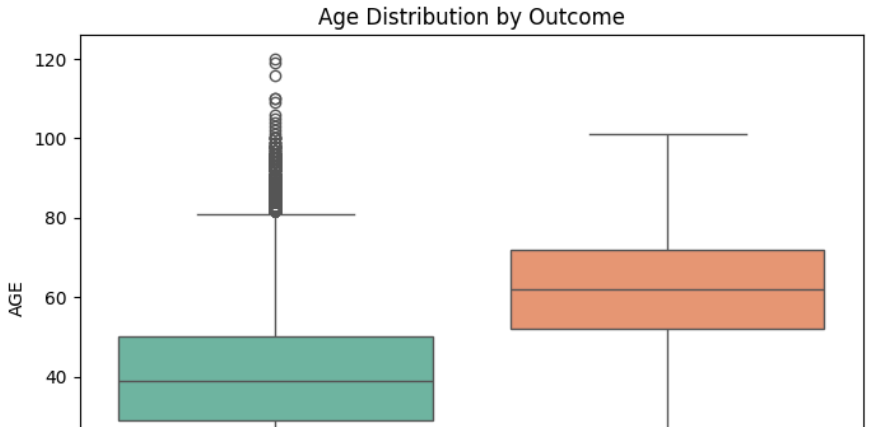
```
# Target Distribution (Count plot showing class frequencies)
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x=target_col, palette='viridis')
plt.title('Target Distribution (DEATH)')
plt.show()

# Feature Distributions and Relationships (At least 4 insightful visualizations)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Age distribution by outcome using a boxplot
sns.boxplot(data=df, x=target_col, y='AGE', ax=axes[0, 0], palette='Set2')
axes[0, 0].set_title('Age Distribution by Outcome')

# Plot 2: Mortality rate by Pneumonia status
sns.barplot(data=df, x='PNEUMONIA', y=target_col, ax=axes[0, 1], ci=None)
axes[0, 1].set_title('Mortality Rate by Pneumonia')

# Plot 3: Age density across genders for both outcomes
sns.violinplot(data=df, x=target_col, y='AGE', hue='SEX', split=True, ax=axes[1, 0])
axes[1, 0].set_title('Age and Sex Distribution by Outcome')
```

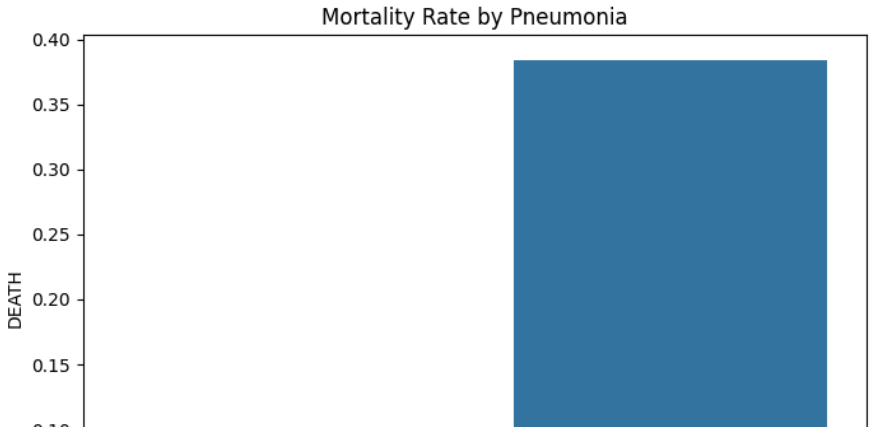
```
# Plot 4: Mortality rate by Hypertension
sns.barplot(data=df, x='HYPERTENSION', y=target_col, ax=axes[1, 1], ci=None)
axes[1, 1].set_title('Mortality Rate by Hypertension')

plt.tight_layout()
plt.show()

# Correlation Heatmap
plt.figure(figsize=(8, 6))
corr_matrix = df[feature_cols + [target_col]].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", center=0)
plt.title('Correlation Heatmap')
plt.show()
```

```
/tmp/ipykernel_219/3469321367.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for t

  sns.countplot(data=df, x=target_col, palette='viridis')
```

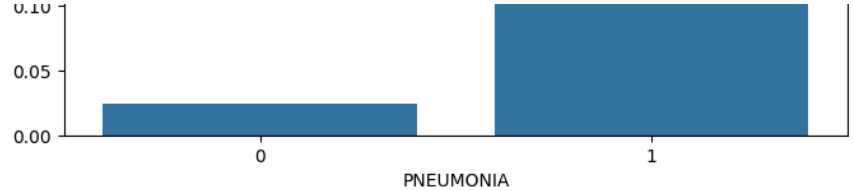


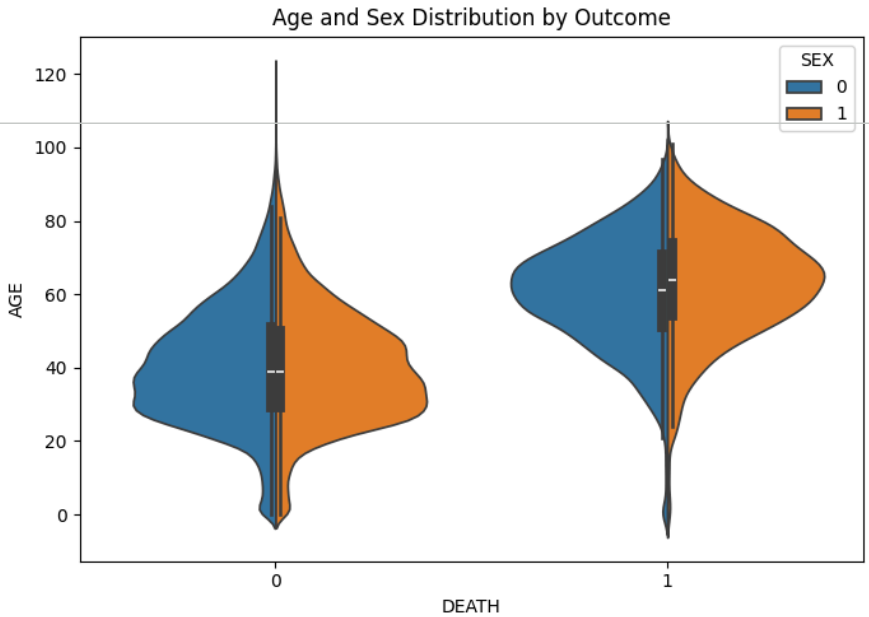

```
/tmp/ipykernel_219/3469321367.py:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for t

  sns.boxplot(data=df, x=target_col, y='AGE', ax=axes[0, 0], palette='Set2')
/tmp/ipykernel_219/3469321367.py:15: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(data=df, x='PNEUMONIA', y=target_col, ax=axes[0, 1], ci=None)
/tmp/ipykernel_219/3469321367.py:23: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(data=df, x='HYPERTENSION', y=target_col, ax=axes[1, 1], ci=None)
```

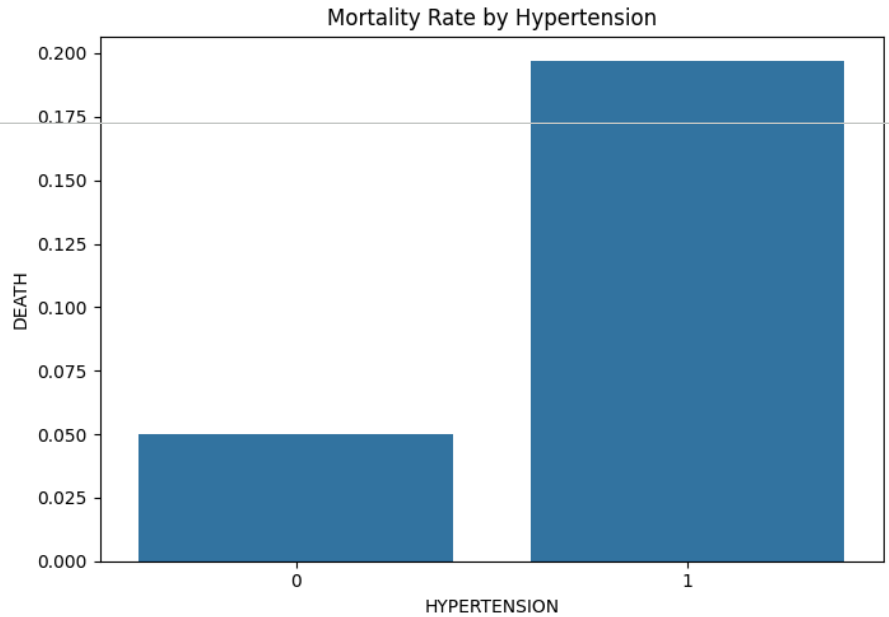


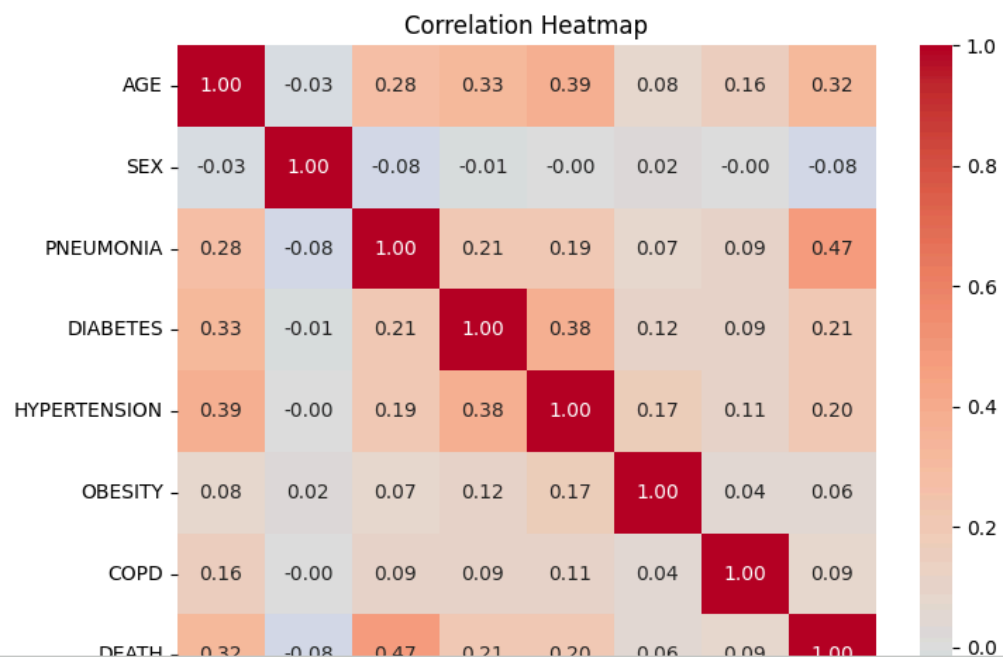


20
0
0
1
DEATH


0.10
0.05
0.00
0
1
PNEUMONIA


Age and Sex Distribution by Outcome
SEX
0
1
120
100
80
60
40
20
0
AGE
0
1
DEATH


Mortality Rate by Hypertension
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
DEATH
0
1
HYPERTENSION


Correlation Heatmap
AGE 1.00 -0.03 0.28 0.33 0.39 0.08 0.16 0.32
SEX -0.03 1.00 -0.08 -0.01 -0.00 0.02 -0.00 -0.08
PNEUMONIA 0.28 -0.08 1.00 0.21 0.19 0.07 0.09 0.47
DIABETES 0.33 -0.01 0.21 1.00 0.38 0.12 0.09 0.21
HYPERTENSION 0.39 -0.00 0.19 0.38 1.00 0.17 0.11 0.20
OBESITY 0.08 0.02 0.07 0.12 0.17 1.00 0.04 0.06
COPD 0.16 -0.00 0.09 0.09 0.11 0.04 1.00 0.09
DEATH 0.32 -0.08 0.47 0.21 0.20 0.06 0.09 1.00
1.0
0.8
0.6
0.4
0.2
0.0

```
# Data Preparation: Train/test split (70/30) and scaling
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

metrics_results = []

def evaluate_model(name, y_true, y_pred, y_prob):
    # Calculate required metrics: Accuracy, Precision, Recall, F1, AUC-ROC
    metrics = {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1": f1_score(y_true, y_pred),
        "AUC-ROC": roc_auc_score(y_true, y_prob)
    }
    metrics_results.append(metrics)
    return metrics

# Logistic Regression Baseline
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
evaluate_model("Logistic Regression", y_test, lr.predict(X_test_scaled), lr.predict_proba(X_test_scaled)[:, 1])

# Decision Tree with GridSearchCV
dt_params = {'max_depth': [3, 5, 7, 10], 'min_samples_leaf': [5, 10, 20]}
dt = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params, cv=5, scoring='f1')
dt.fit(X_train_scaled, y_train)
evaluate_model("Decision Tree", y_test, dt.predict(X_test_scaled), dt.predict_proba(X_test_scaled)[:, 1])

# Random Forest with GridSearchCV
rf_params = {'n_estimators': [50, 100], 'max_depth': [3, 5, 8]}
rf = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=5, scoring='f1', n_jobs=-1)
rf.fit(X_train_scaled, y_train)
evaluate_model("Random Forest", y_test, rf.predict(X_test_scaled), rf.predict_proba(X_test_scaled)[:, 1])

# XGBoost with GridSearchCV (using T4 GPU acceleration)
xgb_params = {'n_estimators': [50, 100], 'max_depth': [3, 5], 'learning_rate': [0.05, 0.1]}
xgb = GridSearchCV(XGBClassifier(random_state=42, eval_metric='logloss'), xgb_params, cv=5, scoring='f1')
xgb.fit(X_train_scaled, y_train)
evaluate_model("XGBoost", y_test, xgb.predict(X_test_scaled), xgb.predict_proba(X_test_scaled)[:, 1])

# Neural Network (MLP) architecture and training
mlp = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])
mlp.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
mlp.fit(X_train_scaled, y_train, epochs=20, batch_size=256, validation_split=0.2, verbose=0)

mlp_prob = mlp.predict(X_test_scaled).flatten()
evaluate_model("Neural Network", y_test, (mlp_prob > 0.5).astype(int), mlp_prob)
```